

What is a machine-learning model?

$$y = f_{\theta}(x)$$

- **x**: input representation
- **f**: model architecture and computation flow
- **θ** : learned parameters
- **y**: output prediction
- **Training vs. Inference**: Training learns θ . Today's focus is inference only: θ is frozen and we evaluate f_{θ} (note: GPT-2 is decoder-only).

$$L = -\log p_{\text{correct}} \quad \theta \leftarrow \theta - \eta \nabla_{\theta} L$$

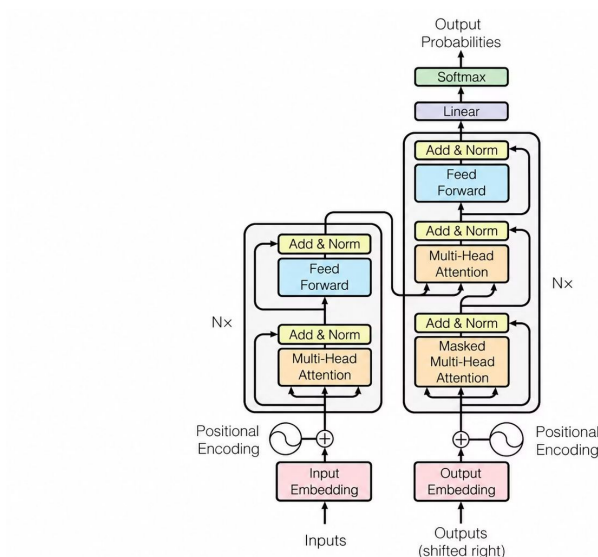
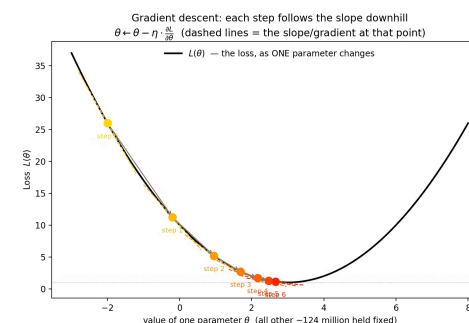


Figure 1: The Transformer - model architecture.



GPT-2 Small: what we run

GPT-2 is compact enough to expose the complete computation pipeline on CPU and GPU without distributed complexity.

Execution pipeline: tokens → embeddings → 12 blocks → ln_f → logits

SPECIFICATION	GPT-2 SMALL CONFIGURATION
Parameters	124M
Transformer blocks	12 layers
Hidden size	768 dimensions
Attention heads	12 heads × 64 dimensions
Vocabulary	50,257 tokens (byte-level BPE)
Context window	1,024 tokens

Scale reference: 124M parameters vs. modern massive models with 100B+ parameters (educational baseline, not a quality ranking).

Text → tokens → vectors

The tokenizer maps text to discrete token IDs; embedding lookup turns IDs and sequence positions into dense vectors.

01	02	03	04
Raw string input	Tokenizer (BPE)	Embedding lookup	Input tensor
"The strangest thing I found inside my refrigerator was"	Fixed mapping produces a sequence of token IDs $[t_0, t_1, \dots, t_T]$	Token IDs index into learned weight matrices	Yields matrix $X \in \mathbb{R}^{T \times 768}$ ready for block 0

$$x_i = W_{te}[t_i] + W_{pe}[i]$$

W_te: Token embeddings ($50,257 \times 768$)

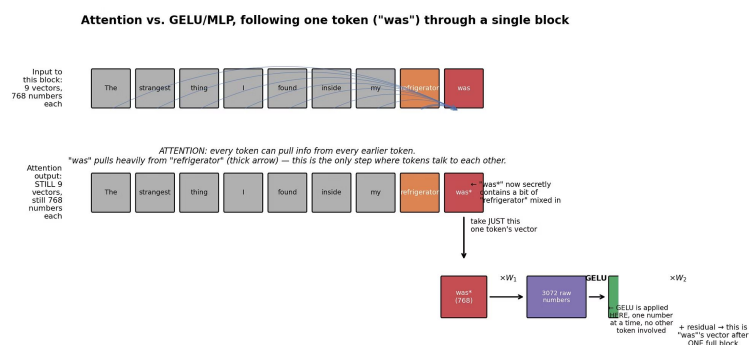
W_pe: Position embeddings ($1,024 \times 768$)

Tensor shape: $T \times 768$

What does a Transformer block actually do?

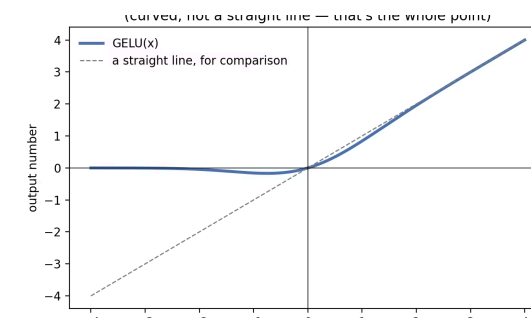
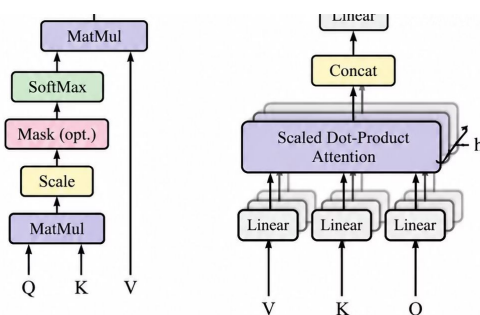
A block alternates between communication across tokens and independent computation within each token.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_h}} + M \right) V$$



1. Attention (Mix tokens): Moves information across positions.

"was" attends back to "refrigerator" so its vector absorbs context. Causal mask M prevents looking into the future.



2. MLP (Per token): Transforms each token independently without mixing. Expands $768 \rightarrow 3072$, applies GELU nonlinearity, then compresses $3072 \rightarrow 768$.

From Transformer math to computer operations

High-level attention equations compile down to four fundamental computer primitives: GEMMs, reductions, elementwise ops, and memory transfers.

Transformer $\approx$ GEMMs + reductions + elementwise math + memory movement

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

TRANSFORMER OPERATION	HARDWARE PRIMITIVE
Q / K / V & Output Projections	Matrix multiplication (GEMM)
QK^T and Attention $\times$ V	Matrix multiplication (GEMM)
Softmax	Reductions (max, sum) + elementwise exp
LayerNorm	Reductions (mean, var) + elementwise scale/bias
Residual addition	Elementwise add
Embedding lookup	Indexed memory read / gather

Our NumPy reference implementation (cpu/gpt2.py)

A line-by-line Python reference that mirrors the exact mathematics and serves as the ground-truth oracle for GPU verification.

Equation → NumPy → future CUDA

kernels: Clean 1-to-1 correspondence between mathematical tensor operations and executable kernels.

```
x = w.wte[token_ids] + w.wpe[pos]
```

```
qkv = linear(x, c_attn_w, c_attn_b)
```

```
q, k, v = np.split(qkv, 3, axis=-1)
```

```
scores = (qh @ kh.transpose(0, 2, 1)) * scale
```

```
attn = softmax(scores, axis=-1)
```

```
out = attn @ v
```

```
h = layer_norm(x); h = causal_self_attention(h); x = x + h
```

```
h = layer_norm(x); h = gelu_new(linear(h, ...)); x = x + h #
```

MLP

```
x = final_layer_norm(x)
```

```
logits = x @ w.wte.T
```

Logits, softmax, and decoding

The forward pass produces raw scores (logits) across the 50,257-token vocabulary; softmax normalizes them into probabilities for token selection.

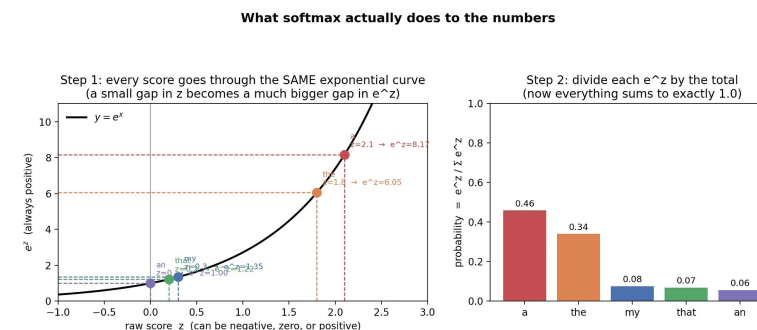
$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Pipeline: 50k logits → softmax → probability distribution → decoding strategy → next token

Decoding strategies: Greedy arg max vs. Top-k / Top-p stochastic sampling.

Key takeaway:

model inference $\neq$ decoding strategy



Q, K, V: no KV cache (demo) vs. KV cache (production)

Autoregressive decoding generates one token at a time. Caching past Keys and Values avoids recalculating prior token representations.

Single linear projection: Generates Q, K, V vectors at each layer.

Our run.py (No KV cache)

Full forward pass every token, recomputing all past K and V ($O(T^2)$ total work). Directly tied to `--verbose-timing` in our CPU benchmark.

Production KV cache

Stores previous K and V in GPU VRAM; only new query Q_{new} attends over the stored history ($O(T)$ per step).

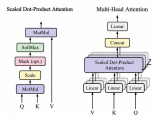


Figure 2: (Left) Single-Head Attention. (Right) Multi-Head Attention consists of several attention heads working in parallel.

Why GPU + CUDA execution: mapping GEMMs to silicon

CPUs optimize for low-latency serial execution; GPUs dedicate silicon to massive SIMT parallel throughput for independent matrix tiles.

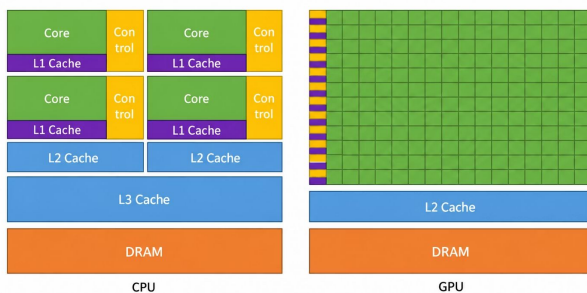


Figure 1: The GPU Devotes More Transistors to Data Processing

GEMM parallelism: Each $C_{ij} = \sum_k A_{ik} B_{kj}$ is computed independently across thousands of GPU cores.

CUDA execution hierarchy: Kernel launch → Grid → Thread Blocks → Warps (32 threads) → Individual Threads.

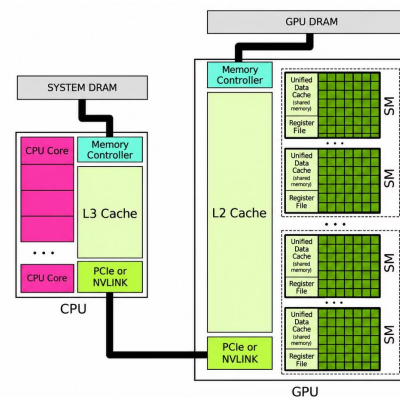


Figure 2: A GPU has many streaming multiprocessors (SMs), each of which contains many functional units. Graphics processing clusters (GPCs) are collections of SMs. A GPU is a set of GPCs connected to a set of DRAMs through a memory controller. A CPU is connected to a GPU through a PCIe bus or NVLINK and also has its own DRAM connected through a memory controller.

Memory hierarchy & tiling: Output matrices are broken into tiles; threads cooperate via registers, shared memory, and L1/L2 caches to minimize global DRAM traffic.

CUDA-Oxide note: Targets these same hardware kernels via Rust and NVIDIA's `cargo oxide` toolchain.

Same math, three runtimes + live CPU demo

Same GPT-2 weights, same mathematical operations, implemented across three execution systems to verify numerical logits and measure performance.

RUNTIME	EXECUTION UNIT	PREFILL LATENCY	DECODE THROUGHPUT	P50 LATENCY	P95 LATENCY
NumPy reference	CPU (x86_64)	Live measured	Live tok/s	Live measured	Live measured
CUDA C++	GPU (NVCC)	TBD	TBD tok/s	TBD	TBD
CUDA-Oxide	GPU (Rust SIMT)	TBD	TBD tok/s	TBD	TBD

```
# Live demonstration trace:
make verify          # Assert logits match within tolerance (logits_CPU ≈ logits_GPU)
python run.py --prompt "The strangest thing I found inside my refrigerator was" --verbose-timing
```